

Le chantier P1 en entier

Force Tracker — 09/09/2026 — de **ft-v1183** a **ft-v1186**, avec le code. Redige pour GPT, qui a ecrit la specification P1 et l'a relue deux fois.

CE QUE COUVRE CE DOCUMENT

Quatre versions, deux relectures, et un modele de donnees qui a change trois fois. Le probleme de depart tenait en une ligne : un **x2** sur une portion de 300 kcal enregistrait **q:null, u:null, per100:null** avec 600 kcal — « 2 portions de 300 » se fossilisait en « 1 portion de 600 », et plus rien ne pouvait redimensionner cette ligne. A l'arrivee, une portion a un **nom**, un **poids**, et l'unite choisie survit a la reprise.

COMMENT CE DOCUMENT EST FABRIQUE — et pourquoi ca compte

Le code ci-dessous n'est pas recopie : il est **EXTRAIT** du depot a chaque generation. Recopier du code dans un document fabrique une deuxieme source de verite qui se perime en silence — le document finirait par montrer autre chose que ce qui tourne. Les commentaires sont retires (ils contiennent des emoji, qui ne se rendent pas dans ce format) ; **le code nu est exact, caractere pour caractere**. Les chiffres viennent des journaux de passe, jamais de la memoire de qui ecrit.

1. La chronologie, et ce que chaque etape a change

Etape	Ce qui a ete fait	Ce que ca a coute
ft-v1183 « portion » devient une unite	Le multiplicateur de portions n'avait aucun propriétaire : <code>_afApplyPortion</code> ne faisait que reecrire les 4 champs a l'ecran. <code>_provFood</code> n'avait aucune branche pour l'unite « portion ».	La passe complete a refuse mon premier correctif : 7 temoins rouges, tous disant « on n'invente pas une quantite qu'elle n'a jamais eue ».
Relecture 1 (Michel)	« 1 portion = 300 kcal, poids inconnu ne suffit pas — je veux savoir si c'est 1 steak, 1 yaourt, 1 dose. »	Mon refus de <code>portionWeightG</code> tombe : je l'avais justifie par une mesure du comportement actuel .
Reponse de conception	Cinq questions repondues sans ecrire une ligne de code, dont l'arrondi de la reference mesure et non suppose.	Ma premiere mesure d'arrondi etait fausse : elle ne testait pas la fraction.
ft-v1186 la portion nommee	Etiquette et poids d'une portion stockes ; masse totale et reference restent derivees .	Un rouge trouve avant livraison : la donnee etait juste et l'ecran mentait .

2. Le modele de donnees, avant et apres

Champ	Avant le chantier	Apres
kcal / prot / carbs / fat	les totaux consommes	inchange
q	null des qu'on comptait en portions	le nombre de portions (2)
u	null	'portion'
portionLabel	<i>n'existait pas</i>	le nom d'UNE portion ('steak')
portionWeightG	<i>n'existait pas</i>	le poids d'UNE portion (125)
per100	null, et non calculable	derive de <code>q x portionWeightG</code>
masse totale	—	jamais stockee — calculee a l'affichage
reference d'une portion	—	jamais stockee — totaux / q

DEUX CHOSES NE SONT PAS STOCKEES, ET C'EST DELIBERE

La **masse totale** et la **reference d'une portion** sont des **vues** de totaux, q et portionWeightG. Stocker une vue cree une seconde source qu'il faut arrondir pour l'ecrire — et c'est l'arrondi qui perd de l'information, pas le calcul. **Mesure** sur 601 kcal / 3 portions, dix cycles de reprise : reference derivee = 200,3333 stable, totaux **601 inchanges** ; reference stockee arrondie = **600 des le premier cycle**, puis fige.

3. Le code, piece par piece

3.1 — Les proprietaires : une variable par notion

Quatre variables, et la separation entre les deux dernieres est la contrainte structurelle de tout le chantier.

app.js — les proprietaires

```
let _afPortions=1;
let _afPortionPose=false;
let _afPortionLabel='', _afPortionPoids=0;
```

Variable	Signifie	Pourquoi elle existe
_afPortions	le multiplicateur choisi (x2)	En grammes, la quantite affichee vit dans le champ af-prop du DOM. En portions, elle ne vivait nulle part.
_afPortionPose	la personne a choisi	Le multiplicateur vaut 1 par default : sans ce drapeau, « jamais touche » et « x1 choisi » sont indiscernables.
_afPortionLabel	le nom d'UNE portion	« 2 portions » ne dit pas de quoi.
_afPortionPoids	le poids d'UNE portion	N'est PAS _afPoidsDeclare, qui est le poids de ce qui est affiche (le total). Deux notions, deux variables, jamais la meme.

Toutes retombent ensemble, sur le meme chemin de remise a zero :

```
function _afResetUnite(){ _afUnite='portion'; _afPoidsDeclare=0; _afPoidsPose=false; _afQtyNom='';
» _afQtyMemo=null; _afPortions=1; _afPortionPose=false; _afPortionLabel=''; _afPortionPoids=0; }
```

3.2 — Le geste : cliquer un multiplicateur

```
function _afApplyPortion(x){
  _afPortions=(+x>0)?x:1;
  _afPortionPose=true;
  if(typeof _afMajAncre==='function') _afMajAncre();
  _afProp(_afPortions);
}
```

Trois lignes, et chacune repare un default distinct : la premiere **retient** le choix (il n'etait ecrit nulle part), la deuxieme **l'affirme** (jumeau de _afPoidsPose), et _afMajAncre() est appele **sans** srcChange — il redessine, il ne relit pas l'ecran, sinon la reference deviendrait les valeurs deja multipliees et l'erreur se figerait.

3.3 — L'invariant : quand l'ecran redevient la reference

app.js — dans _afMajAncre

```
const relit=(srcChange||!_afRef||!_afRef.base);
const base=relit?_afPropSetBase():_afRef.base;
if(relit){ _afPortions=1; _afPortionPose=false; }
```

Mesure : on tape x2 (ecran 600), puis on corrige une macro a la main. La source a change, donc base **devient 600**. Garder _afPortions=2 enregistrerait « 2 portions » pour des totaux qui sont deja ceux de deux portions. *Ce qui est affiche est, par definition, UNE portion de lui-meme.*

3.4 — La definition, et ce qu'elle refuse d'inventer

```
function _portionDefTexte(base,n, label,poids){
  const b=base||{}; q=(+n>0)?+n:1; p=+poids||0;
  const nom=String(label||'').trim();
  const k=Math.round(+b.kcal||0);
  const val=(k>0) ? k+' kcal'
    : 'P '+Math.round(+b.prot||0)+' · G '+Math.round(+b.carbs||0)+' · L '+Math.round(+b.fat||0);
  const esc = (typeof _escFood==='function') ? _escFood(nom) : nom;
  const quoi = nom ? ('1 '+esc) : '<i>portion non définie</i>';
  const masse = (p>0) ? (' '+_nbFR(p)+' g') : ', poids inconnu';
  const def='1 portion = '+quoi+masse+' · '+val;
  const tete = (q===1) ? 'Les 4 valeurs ci-dessous sont <b>1 portion</b>'
    : 'Tu notes <b>'+_portionLbl(q)+' portions</b>';
  const total = (p>0 && q!=='') ? ' Soit <b>'+_nbFR(Math.round(q*p))+' g</b> en tout.' : '';
  return tete+' ('+def+').'+total;
}
```

Un **seul propriétaire** du texte, lu par l'écran d'ajout **et** par celui d'édition : deux copies finiraient par ne plus dire la même chose. Sans nom -> « portion non définie ». Sans poids -> « poids inconnu ». **La masse totale est calculée ici et nulle part ailleurs**, et seulement si elle est connue.

3.5 — La saisie : pourquoi la frappe ne redessine jamais

```
function _afPortionNom(m){
  const v=String(m||'').trim().slice(0,24);
  _afPortionLabel = (_afPortionLabel.toLowerCase()===v.toLowerCase()) ? '' : v;
  _afPortionPose=true;
  if(typeof _afMajAncre==='function') _afMajAncre();
  _afProp(_afPortions>0?_afPortions:1);
}
function _afMajDefPortion(){
  const el=document.getElementById('af-pdef');
  if(el && _afRef && _afRef.base) el.innerHTML=_portionDefTexte(_afRef.base,_afPortions,_afPortionLabel,_afPortionPoids);
}
function _afPortionNomSaisi(){
  _afPortionLabel=String((document.getElementById('af-pnom')||{}).value||'').trim().slice(0,24);
  if(_afPortionLabel) _afPortionPose=true;
  _afMajDefPortion();
}
function _afPortionPoidsSaisi(){
  const v=numFR((document.getElementById('af-ppoids')||{}).value);
  _afPortionPoids=(v>0 && v<10000)?v:0;
  if(_afPortionPoids>0) _afPortionPose=true;
  _afMajDefPortion();
}
```

Une **puce** est un geste unique : on peut redessiner (il faut l'allumer). Une **frappe**, non — reconstruire le bloc pendant qu'on tape dedans **detruit le champ au premier caractère**. Les deux fonctions de saisie écrivent la variable et ne rafraichissent que la ligne de définition.

ET LE POIDS D'UNE PORTION NE RESCALE RIEN — c'est toute la différence avec l'autre champ

Savoir qu'un steak pèse 125 g **ne change pas ce qu'on a mangé** : deux steaks restent deux steaks. Le champ de poids de l'onglet grammes, lui, **cale** les valeurs. Ce champ-ci n'est qu'un **pont vers la masse** : il permet d'afficher « 250 g en tout » et de calculer un pour-100 g. **Et il ne bascule pas l'unité** — c'était la demande n°2 de Michel, mot pour mot. Deux témoins figent la distinction (mesure : 240 / 16 avant et après).

3.6 — La descente jusqu'à la donnée

app.js — dans _provFood, la branche portion

```
if(!p.q && _afPortionPose && _afUnite==='portion' && typeof _afRef==='object' && _afRef && _afRef.u==='') &&
  » vals){
  const n=(typeof _afPortions==='number' && _afPortions>0)?_afPortions:1;
  p.q=n; p.u='portion';
  if(_afPortionLabel) p.portionLabel=String(_afPortionLabel).slice(0,24);
  if(+_afPortionPoids>0) p.portionWeightG=+_afPortionPoids;
  const masse=n*(+_afPortionPoids||0);
  if(!p.per100 && masse>0){
    const f=100/masse;
    p.per100={kcal:Math.round((+vals.kcal||0)*f), prot:Math.round((+vals.prot||0)*f),
      carbs:Math.round((+vals.carbs||0)*f), fat:Math.round((+vals.fat||0)*f)};
  }
}
```

Le garde sur `_afUnite` **n'est pas decoratif, et la mesure l'a exige** : l'onglet grammes, *avant* qu'un poids soit declare, pose lui aussi `_afRef={q:1,u:' '}`. Sans ce test, quelqu'un qui hesite sur cet onglet verrait sa ligne enregistree **en portions** — une unite qu'il n'a pas choisie.

Et le pour-100 g est **recalcule a chaque ecriture** depuis `q x portionWeightG`, jamais traite comme une verite independante — c'est l'option (a) trantee par Michel. On n'ecrase jamais un pour-100 g deja connu : une declaration ne passe pas devant une valeur publiee.

3.7 — La liste blanche, oubliee pour la quatrieme fois

app.js — dans `_provFood`, la partie qui recopie la source

```
else if(+_afSrc.q>0 && _afSrc.u==='portion'){ p.q+=_afSrc.q; p.u='portion'; }
if(!_afSrc.portionLabel) p.portionLabel=String(_afSrc.portionLabel).slice(0,24);
if(+_afSrc.portionWeightG>0) p.portionWeightG+=_afSrc.portionWeightG;
```

LE MEME OUBLI, QUATRE FOIS, AU MEME ENDROIT

Cette fonction construit une **liste blanche** : un champ pose en amont et non recopie ici **n'atteint jamais l'entree enregistree — sans erreur, sans test rouge**. L'avertissement est ecrit **trois fois en majuscules** dans cette meme fonction, et c'est arrive une quatrieme fois. *Un avertissement en commentaire ne protege de rien. Cette fois un temoin dedie fige la traversee.*

3.8 — La reprise : l'unite appartient a la personne

```
function _afReprendrePortions(n){
  const q=(+n>0)?n:1;
  if(!_afRef||!_afRef.base) return;
  _afUnite='portion'; _afPoidsDeclare=0;
  const b=_afRef.base;
  _afRef.base={kcal:(+b.kcal||0)/q, prot:(+b.prot||0)/q, carbs:(+b.carbs||0)/q, fat:(+b.fat||0)/q};
  _afRef.q=1; _afRef.u=''; _afRef.src='portion';
  _afApplyPortion(q);
}
```

L'entree porte les **totaux** et `q` ; la reference d'une portion est donc `totaux / q` — on ne l'invente pas, on la retrouve. Aucun etat d'ecran nouveau : on repose le bloc existant avec le bon multiplicateur allume.

app.js — dans `quickFillFood` (la porte jumelle `_afSuggPrendreLocale` a les memes)

```
if(it.u==='portion'){ _afPortionLabel=String(it.portionLabel||'').slice(0,24);
  _afPortionPoids+=it.portionWeightG>0?it.portionWeightG:0; }
if(!_bcNutr && +it.q>0 && it.u==='portion' && typeof _afReprendrePortions==='function')
  » _afReprendrePortions(+it.q);
if(P && it.u!=='portion' && (+P.kcal>0 || +P.prot>0 || +P.carbs>0 || +P.fat>0)){
```

3.9 — Le favori : rafraichir la definition, jamais les macros

```
function _majDefFavori(e){
  try{
    if(!e || e.u!=='portion' || !Array.isArray(S.savedFoods)) return;
    const k=String(e.name||'').toLowerCase(); if(!k) return;
    » const f=S.savedFoods.find(x=>String(x.name||'').toLowerCase()===k); if(!f) return;
    if(e.portionLabel && f.portionLabel!==e.portionLabel) f.portionLabel=e.portionLabel;
    if(+e.portionWeightG>0 && +f.portionWeightG!+=e.portionWeightG) f.portionWeightG+=e.portionWeightG;
  }catch(err){}
}
```

« *Un steak pese 125 g* » est un fait sur l'**aliment** ; « *j'en ai mange 2* » est un fait sur **ce repas-la**. Ecraser les macros ferait qu'un gros repas redefinirait le favori pour toujours. Et une definition vide n'efface jamais l'ancienne.

3.10 — L'export

setup.js — deux colonnes de plus

```
const NUTRI_COLONNES = ['date','repas','aliment','quantite','unite','portion_label','portion_poids_g','kcal','
» proteines_g','glucides_g','lipides_g','saisie','source'];
  portion_label: e && e.portionLabel || '',
  portion_poids_g: (e && +e.portionWeightG > 0) ? e.portionWeightG : '',
```

Sans elles, l'export perdrait exactement l'information qu'on vient de sauver.

4. Les trois fois ou la mesure a arrete un correctif

C'est la partie la plus utile de ce document, parce qu'aucune de ces trois erreurs n'était visible a la relecture.

Ce que j'allais livrer	Ce que la mesure a dit	La regle qui en sort
Ecrire q:1, u: 'portion' des que le bloc portions est affiche	7 temoins rouges , tous « on n'invente pas une quantite qu'elle n'a jamais eue ». Le bloc portions est l'etat par default : l'afficher ne prouve aucun choix.	Il fallait un drapeau. Les temoins n'ont pas ete desserres : les deux du nouveau bloc qui attendaient q:1 sont passes a la garantie plus forte, q: null.
Refuser portionWeightG parce que « cet etat n'existe pas »	L'etat n'existe pas dans le comportement actuel . Le besoin, lui, existe : « 2 steaks de 125 g » perd son sens en grammes.	Une mesure du comportement actuel ne justifie jamais un refus de BESOIN . Elle decrit ce qui est, pas ce qui manque.
Livrer la reprise telle quelle	Reprendre « 2 steaks » rouvrant le champ grammes (un pour-100 g existait) et l'ecran perdait le « 2 ». La donnee etait intacte, l'ecran mentait .	Le pour-100 g ne decide plus de l'unite . L'unite appartient a la personne, pas a la richesse de la fiche.

5. Validation

Suite	Resultat
Bloc de temoins de ft-v1183 (CCLXXX)	15 temoins
Bloc de temoins de ft-v1186 (CCLXXXI)	14 temoins
Parcours complet (bout en bout, vrai navigateur)	3451 verts, 0 rouge(s)
Calculs / muscles / croises / dates	339/339 · 241/241 · 50/50 · 9/9
Donnees classees face au moteur conversationnel	0 trou non classe
Controle negatif (ft-v1186)	15 mutations, toutes mordent
Deploiement	run #1054, success a 19:54:25 UTC

UNE PROTECTION SANS TEMOIN N'EST PAS UNE PROTECTION — quatre fois dans ce seul chantier

Quatre protections se sont revelees **sans temoin**, et le controle negatif ne l'a dit que parce qu'on lui a demande : **(1)** l'etat « boutons de portion » de l'ecran d'edition ; **(2)** le drapeau cote edition ; **(3)** le rafraichissement du favori — **et la, ce n'etait pas le temoin qui manquait, c'etait ma FIXTURE** : elle mettait 600 des deux cotes, donc l'ecrasement etait arithmetiquement invisible ; **(4)** le garde-fou de ce document lui-meme (voir plus bas). *Une mutation qui ne mord pas ne prouve pas que le code est inutile : elle pose une question, et il faut y repondre.*

6. Deux temoins d'une version precedente ont ete touches

La passe complete de ft-v1186 a fait rougir **deux temoins de ft-v1183**. Ils exigeaient le motif exact « 1 portion = 300 kcal » ; en ajoutant le nom, la phrase devient « 1 portion = portion non definie, poids inconnu · 300 kcal ».

La garantie qu'ils figeaient est intacte, et meme plus forte : l'ecran annonce desormais aussi que le nom manque. Ce n'est pas un temoin qu'on desserre pour faire passer du code — c'est le motif qui figeait plus que sa garantie. Motifs resserres sur les trois exigences reelles, modification **datee et signee dans le fichier de tests**, et surtout **eprouvee** : retirer la definition, taire le poids inconnu ou supprimer le nombre de portions les fait toujours rougir. *La difference entre une mise au point et un assouplissement se mesure.*

7. Ce qui reste ouvert

Sujet	Etat
Migration des lignes déjà abimées	Intacte, rien n'a été tenté. Une ligne a <code>q:null</code> , <code>per100:null</code> ne permet pas de deviner si 323 kcal valaient 100 ou 300 g. L'outil a trois niveaux (certain / ambigu / insuffisant) reste à construire.
Afficher « 2 steaks » dans le journal du jour	Prochain chantier , décide séparé par Michel. Le journal n'affiche aujourd'hui aucune quantité — ni grammes ni portions.
Le garde-fou de masse en portions	Toujours muet. Avec <code>portionWeightG</code> il pourrait désormais parler.
Retirer une étiquette fautive	Possible seulement en éditant l'entrée, pas depuis l'écran d'ajout. <i>Assume et écrit, pas oublié.</i>
Corriger <code>portionWeightG</code> après coup	Non testé : une ligne dont on changerait le poids de portion depuis l'édition, sans repasser par l'ajout. Si tu vois la un chemin ou le pour-100 g survit à une définition qui a changé, il m'intéresse.
Cru / cuit · recherche CIQUAL · scan	Hors périmètre, non touchés.
Validation iPhone	À faire par Michel : pas de WebKit dans le conteneur.

8. Et ce document a répété la même faute

TROIS EMOJI, DES CARRÉS NOIRS, ET UN GARDE-FOU FAUX

Dans la version précédente de ce fil, trois emoji ont fui dans le texte du PDF et sortaient en **carrés noirs** — alors que l'en-tête du générateur l'annonce noir sur blanc. *Un avertissement en commentaire ne protège de rien ; seul un contrôle qui échoue protège.* **Et ma première version du garde-fou était fautive** : elle n'inspectait que les paragraphes, donc ni les encadres ni les tableaux — éprouvée, elle a **laissé passer** un emoji dans un titre. **Une protection partielle qui a l'air complète est pire qu'une protection absente.** Elle est posée sur les cinq portes par lesquelles le texte entre, et les cinq sont éprouvées. *C'est exactement la leçon du paragraphe 5, commise sur l'outil qui sert à la raconter.*

OU LIRE LA SUITE

Les trois documents précédents de ce fil : `docs/P1-PORTIONS-DEBRIEF.pdf` (ft-v1183), `docs/P1-PORTIONS-CONCEPTION.pdf` (la réponse de conception) et `docs/P1-PORTION-NOMMEE.pdf` (ft-v1186). Le journal de version est dans `CLAUDE.md`, l'état courant dans `docs/CONTEXTE-ACTUEL.md`, les doutes non promus dans `docs/JOURNAL-DE-TEST.md`, et le catalogue des familles de bugs dans `BUGS.md`. Le dépôt est public : github.com/michdu75-commits/forcetracker.